

مستندات توسعه‌دهندگان

راهنمای API پیامک

ارسال پیامک، پیگیری وضعیت، مدیریت مخاطبین و دریافت رویدادها
از طریق REST API پنل هوشمند ELLSMS

نسخه‌ی API	آدرس پایه	تاریخ انتشار
v1	ellsms.ir/api/v1	مهر ۱۴۰۵

فهرست مطالب

اگر عجله دارید، بخش ۱ برای ارسال اولین پیامک کافی است. بخش‌های ۴ و ۵ مرجع کامل ارسال‌اند.

۱	**شروع سریع**	سه قدم تا اولین پیامک	1
۲	**احراز هویت و کلیدها**	کلید API، دسترسی‌ها، محدودیت آی‌پی	1
۳	**انواع ارسال**	کدام روش برای کدام کار	2
۴	**ارسال تکی و گروهی**	مرجع کامل POST /messages	3
۵	**ارسال انبوه و تدریجی**	مرجع کامل POST /bulk-jobs	5
۶	**پیش‌نمایش هزینه**	هزینه را پیش از ارسال ببینید	8
۷	**جلوگیری از ارسال تکراری**	client_message_id و Idempotency-Key	8
۸	**سایر مسیرها**	موجودی، مخاطبین، وبهوک‌ها	9
۹	**وبهوک‌ها**	رویدادها و بررسی امضا	10
۱۰	**محدودیت‌ها**	نرخ درخواست و سقف‌ها	11
۱۱	**خطاها**	قالب خطا و همه‌ی کدها	11
۱۲	**نمونه کد**	Node.js و PHP، Python	13
۱۳	**کتابخانه‌ها و منابع**	SDK، OpenAPI و چک‌لیست	14

پرسش یا مشکلی دارید؟ با شناسه‌ی `request_id` خطا به پشتیبانی ELLSMS پیام دهید.

۱ شروع سریع

با API پیامکِ ELLSMS، نرم‌افزار، فروشگاه یا سامانه‌ی شما بدون ورود به پنل پیامک می‌فرستد، وضعیت تحویل را می‌گیرد و موجودی اعتبار را می‌بیند. همه‌ی درخواست‌ها JSON هستند و از HTTPS ارسال می‌شوند.

۱

کلید بسازید

در پنل، بخش «توسعه‌دهندگان ← کلیدهای API»، یک کلید با دسترسی‌های لازم بسازید.

۲

هدر بگذارید

کلید را در هدر `Authorization` هر درخواست قرار دهید.

۳

بفرستید

یک درخواست `POST` به مسیر `/messages` بزنید و نتیجه را همان لحظه بگیرید.

آدرس پایه

```
https://ellsms.ir/api/v1
```

اولین پیامک شما

CURL

```
curl -X POST https://ellsms.ir/api/v1/messages \
  -H "Authorization: Bearer $ELLSMS_KEY" \
  -H "Content-Type: application/json" \
  -d '{"destinations": ["989121234567"], "content":"سلام! این اولین پیامک من است."}'
```

همین!

اگر پاسخ با کد `201` و `"status": "sent"` برگشت، پیامک ارسال شده است. ادامه‌ی این راهنما همه‌ی جزئیات را توضیح می‌دهد.

۲ احراز هویت و کلیدها

هر درخواست باید یک کلید API معتبر داشته باشد. کلید را فقط در هدر بفرستید؛ کلیدی که در آدرس (query string) بیاید پذیرفته نمی‌شود.

HEADERS

```
Authorization: Bearer ellsms_live_a1b2c3d4e5f6_kQ3fZ8...
Content-Type: application/json
```

کلید فقط یک بار نمایش داده می‌شود

بلافاصله پس از ساخت، آن را در جای امنی (مثل متغیر محیطی سرور) ذخیره کنید. ELLSMS خودِ کلید را نگه نمی‌دارد و قابل بازیابی نیست؛ اگر گم شد، کلید را چرخش (rotate) دهید یا کلید تازه بسازید. هرگز کلید را در کد سمت مرورگر یا اپلیکیشن موبایل قرار ندهید.

دسترسی‌ها (Scope)

هنگام ساخت کلید مشخص می‌کنید آن کلید چه کارهایی مجاز است انجام دهد. بهتر است هر کلید فقط دسترسی‌هایی را داشته باشد که واقعاً لازم دارد.

Scope	اجازه می‌دهد
`messages:send`	ارسال پیامک تکی/گروهی و پیش‌نمایش هزینه‌ی آن
`messages:read`	خواندن وضعیت یک ارسال
`bulk:write`	ساخت ارسال انبوه و پیش‌نمایش هزینه‌ی آن
`bulk:read`	پیگیری وضعیت ارسال انبوه
`contacts:read` / `contacts:write`	خواندن / ساخت، ویرایش و حذف مخاطبین
`balance:read`	مشاهده‌ی موجودی اعتبار
`webhooks:read` / `webhooks:write`	مدیریت وبهوک‌ها

محدودیت آی‌پی (اختیاری)

مالک یا مدیر سازمان می‌تواند در همان صفحه‌ی کلیدها، فهرست آی‌پی‌های مجاز را فعال کند. پس از آن، درخواستی که از آی‌پی دیگری برسد با خطای `403` رد می‌شود — حتی اگر کلید درست باشد. برای سرورهایی با آی‌پی ثابت توصیه می‌شود.

۳ انواع ارسال

بسته به اینکه یک متن را برای چند نفر می‌فرستید یا برای هر نفر متن جداگانه دارید، یکی از این روش‌ها را انتخاب کنید:

فوری

تکی و گروهی

یک متن برای ۱ تا ۱۰۰ شماره. نتیجه‌ی نهایی در همان پاسخ برمی‌گردد. مناسب کد تأیید، اطلاع‌رسانی سفارش و هشدار.

`POST /messages`

در صف

انبوه شخصی‌سازی‌شده

متن متفاوت برای هر شماره، تا ۵۰۰۰ ردیف در هر درخواست. شناسه می‌گیرید و پیشرفت را پیگیری می‌کنید.

`POST /bulk-jobs`

در صف، کنترل‌شده

تدریجی

مثل انبوه، ولی با سرعت مشخص — مثلاً هر ۱۰ دقیقه ۱۰۰ پیامک — تا ترافیک یا پشتیبانی شما یکجا درگیر نشود.

`type: gradual`

اگر…	از این استفاده کنید
یک پیام یکسان برای چند نفر دارید (تا ۱۰۰ نفر)	`POST /messages`
برای هر نفر متن خاص خودش را دارید (نام، مبلغ، کد…)	`POST /bulk-jobs` با `type: p2p`
بیش از ۱۰۰ گیرنده دارید، حتی با متن یکسان	`POST /bulk-jobs`
می‌خواهید ارسال در طول زمان پخش شود	`POST /bulk-jobs` با `type: gradual`

۴ ارسال تکی و گروهی

POST `/api/v1/messages`

یک متن را برای یک یا چند شماره می‌فرستد. این درخواست **همزمان** است: پاسخ، نتیجه‌ی نهایی ارسال را در خود دارد و نیازی به پیگیری جداگانه نیست. دسترسی لازم: `messages:send`

فیلدهای درخواست

فیلد	وضعیت	توضیح
`destinations`	**اجباری**	آرایه‌ای از شماره‌ها (رشته)، حداکثر ۱۰۰ عدد. شکل‌های `0912…` و `98912…` هر دو پذیرفته و خودکار یکسان می‌شوند.
`content`	**اجباری**	متن پیام، حداکثر ۲۰۰۰ نویسه. پیام‌های طولانی خودکار چندبخشی ارسال و به همان اندازه محاسبه می‌شوند.
`originator`	اختیاری	خط ارسال‌کننده. باید یکی از خطوطِ مجازِ صاحب کلید باشد (اختصاصی، سازمانی یا اشتراکی). اگر نفرستید، خط پیش‌فرض پنل استفاده می‌شود.
`channel`	اختیاری	`sms` (پیش‌فرض)، `bale` یا `bale_sms` — جدول پایین.
`client_message_id`	اختیاری	شناسه‌ی یکتای شما برای جلوگیری از ارسال تکراری (بخش ۷).

نمونه

REQUEST

```
curl -X POST https://ellsms.ir/api/v1/messages \
  -H "Authorization: Bearer $ELLSMS_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "originator": "500030898989",
    "destinations": ["989121234567", "09351111111"],
    "content": "سفارش ۱۰۲۴ شما ثبت شد. ممنون از خریدتان.",
    "client_message_id": "order-1024-confirm"
  }'
```

RESPONSE · 201 CREATED

```
{
  "data": {
    "id": "482",
    "status": "sent",
    "sent_count": 2,
    "total_count": 2,
    "channel": "sms"
  }
}
```

وضعیت ارسال

status	کد HTTP	معنی
sent	201	به همه‌ی گیرنده‌ها ارسال شد.
partially_sent	201	به بخشی ارسال شد؛ sent_count و total_count را مقایسه کنید.
failed	422	به هیچ گیرنده‌ای ارسال نشد و اعتباری کم نشد.

> **«ارسال شد» یعنی «تحویل شد» نیست**
>
> sent یعنی پیامک به اپراتور سپرده شد. برای اطلاع از تحویل واقعی به گوشی گیرنده، وبهوک message.delivered را فعال کنید (بخش ۹).

کانال ارسال

channel	رفتار
sms	پیامک معمولی. پیش‌فرض.
bale	فقط از طریق پیام‌رسان بله.
bale_sms	اول بله؛ برای گیرنده‌هایی که بله نپذیرفت، پیامک. پاسخ شامل sent_by_channel است، مثلاً {"bale": 8, "sms": 2} .

کانال‌های بله فقط زمانی در دسترس‌اند که روی سامانه پیکربندی شده باشند.

پیگیری بعدی

GET `/api/v1/messages/{id}`

همان ساختار پاسخ بالا را با شناسه برمی‌گرداند. دسترسی لازم: messages:read

۵ ارسال انبوه و تدریجی

POST `/api/v1/bulk-jobs`

برای هر گیرنده متن خودش را دارید، یا تعدادشان از ۱۰۰ بیشتر است. درخواست در صف قرار می‌گیرد و شما یک شناسه برای پیگیری دریافت می‌کنید. دسترسی لازم: bulk:write

هدر `Idempotency-Key` در این مسیر اجباری است

برای هر کمپین یک مقدار یکتا بسازید (مثلاً `campaign-1405-07-invoices`). اگر درخواست قطع شد و دوباره فرستادید، با همان کلید کمپین دوباره ساخته نمی‌شود.

فیلدهای درخواست

فیلد	وضعیت	توضیح
`type`	**اجباری**	`p2p`، `smart` یا `gradual` — جدول پایین.
`title`	**اجباری**	عنوان کمپین؛ در گزارش‌های پنل با همین نام دیده می‌شود.
`items`	**اجباری**	آرایه‌ای از `{"mobile", "content"}`، حداکثر ۵۰۰۰ ردیف.
`originator`	اختیاری	خط ارسال‌کننده، با همان قاعده‌ی ارسال تکی.
`throttle_count`	فقط gradual	تعداد پیامک در هر نوبت (عدد صحیح ≥ ۱).
`throttle_minutes`	فقط gradual	فاصله‌ی بین نوبت‌ها به دقیقه (عدد صحیح ≥ ۱).

مقادیر type

type	رفتار
`p2p`	ارسال نظیربه‌نظیر: هر ردیف با متن خودش، با بیشترین سرعت ممکن.
`smart`	از نظر ارسال دقیقاً مانند `p2p` است؛ فقط در گزارش‌ها با برچسب «پیامک هوشمند» نمایش داده می‌شود.
`gradual`	ارسال تدریجی: هر `throttle_minutes` دقیقه، `throttle_count` پیامک.

نمونه

REQUEST

```
curl -X POST https://ellsms.ir/api/v1/bulk-jobs \
  -H "Authorization: Bearer $ELLSMS_KEY" \
  -H "Content-Type: application/json" \
  -H "Idempotency-Key: campaign-1405-07-invoices" \
  -d '{
    "type": "p2p",
    "title": "یادآوری فاکتور مهر",
    "originator": "500030898989",
    "items": [
      {"mobile": "989121234567", "content": "آقای رضایی، مبلغ فاکتور شما ۲۵۰٬۰۰۰ تومان است."},
      {"mobile": "989351111111", "content": "خانم احمدی، مبلغ فاکتور شما ۴۸۰٬۰۰۰ تومان است."}
    ]
  }'
```

RESPONSE · 201 CREATED

```
{ "data": { "id": "91", "status": "pending", "total_rows": 2 } }
```

نمونه‌ی تدریجی

BODY

```
{
  "type": "gradual",
  "title": "اطلاع‌رسانی جشنواره",
  "throttle_count": 100,
  "throttle_minutes": 10,
  "items": [ ... ]
}
```

پیگیری پیشرفت

GET `/api/v1/bulk-jobs/{id}`

RESPONSE · 200

```
{
  "id": "91", "type": "p2p", "title": "یادآوری فاکتور مهر",
  "status": "processing",
  "sent_rows": 1240, "failed_rows": 3, "total_rows": 5000,
  "created_at": "2026-10-04T09:30:00+03:30"
}
```

به‌جای پرسیدن مکرر، می‌توانید وبهوک `bulk.completed` را فعال کنید تا پایان کار به شما اطلاع داده شود. دسترسی لازم: `bulk:read`

۶ پیش‌نمایش هزینه

پیش از ارسال واقعی، می‌توانید **همان بدنه‌ی درخواست** را به مسیر پیش‌نمایش بفرستید تا هزینه را ببینید:

POST `/api/v1/messages/preview`

POST `/api/v1/bulk-jobs/preview`

پاسخ شامل تعداد گیرنده‌های معتبر، تعداد بخش‌های هر پیام، هزینه‌ی تخمینی، موجودی پیش و پس از ارسال و اثر آن بر سهمیه است.

> **پیش‌نمایش هیچ چیزی را تغییر نمی‌دهد**
>
> نه پیامی ارسال می‌شود، نه اعتباری رزرو می‌شود و نه از سهمیه کم می‌شود. هرچند بار که بخواهید می‌توانید آن را صدا بزنید و به `Idempotency-Key` هم نیازی ندارد.

پیش‌نمایش تخمینی است: هنگام ارسال واقعی، هزینه دوباره محاسبه و همان لحظه رزرو می‌شود. اگر در این فاصله موجودی کافی نباشد، ارسال با خطا متوقف می‌شود — هرگز بیش از موجودی برداشت نمی‌شود.

۷ جلوگیری از ارسال تکراری

شبکه همیشه قابل اعتماد نیست. اگر درخواست شما با خطای زمان‌انتظار (timeout) قطع شد، نمی‌دانید پیامک رفته یا نه. ارسال دوباره‌ی بی‌گدار ممکن است به مشتری دو پیامک برساند و دو بار هزینه بگیرد.

راه‌حل: به هر ارسال یک شناسه‌ی یکتا از طرف خودتان بدهید — مثلاً شماره‌ی سفارش:

```
BODY
{ "client_message_id": "order-1024-confirm", "destinations": [...], "content": "..." }
```

یا همان مقدار را در هدر `Idempotency-Key` بفرستید. سپس:

اگر دوباره بفرستید...	نتیجه
با همان شناسه و همان بدنه	همان پاسخ قبلی برمی‌گردد. **پیامکی دوباره ارسال نمی‌شود.**
با همان شناسه ولی بدنه‌ی متفاوت	خطای `409 conflict` — شناسه را اشتباهاً تکرار کرده‌اید.
وقتی درخواست اول هنوز در جریان است	خطای `409` — کمی بعد دوباره امتحان کنید.

این شناسه برای `/messages` تا ۲۴ ساعت معتبر است.

۸ سایر مسیرها

مسیر	دسترسی	کاربرد
GET `/me`	—	اطلاعات کلید فعلی و دسترسی‌هایش. برای تست اتصال عالی است.
GET `/organization`	—	اطلاعات سازمان صاحب کلید.
GET `/balance`	`balance:read`	موجودی اعتبار.
GET `/messages/{id}`	`messages:read`	وضعیت یک ارسال تکی/گروهی.
GET `/bulk-jobs/{id}`	`bulk:read`	پیشرفت یک ارسال انبوه.
GET POST `/contacts`	`contacts:*`	فهرست و ساخت مخاطب.
GET PATCH DELETE `/contacts/{id}`	`contacts:*`	خواندن، ویرایش و حذف مخاطب.
GET POST `/webhooks`	`webhooks:*`	فهرست و ساخت وبهوک.
GET PATCH DELETE `/webhooks/{id}`	`webhooks:*`	مدیریت یک وبهوک.
POST `/webhooks/{id}/rotate-secret`	`webhooks:write`	تعویض کلید امضای وبهوک.
POST `/webhooks/{id}/test`	`webhooks:write`	ارسال یک رویداد آزمایشی.

نمونه: موجودی اعتبار

```
GET /BALANCE · 200

{ "data": { "available": 6876380, "reserved": 12500, "total": 6888880, "unit": "credits" } }
```

`reserved` اعتباری است که برای ارسال‌های در جریان کنار گذاشته شده و پس از پایان آن‌ها یا کم می‌شود یا برمی‌گردد.

۹ وبهوک‌ها

به‌جای اینکه مدام وضعیت را بپرسید، ELLSMS هر اتفاق مهم را با یک درخواست `POST` به آدرس شما خبر می‌دهد. آدرس باید `HTTPS` باشد.

رویدادها

رویداد	زمان ارسال
`message.sent`	پیامک به اپراتور سپرده شد.
`message.failed`	ارسال پیامک ناموفق بود.
`message.delivered`	پیامک به گوشی گیرنده رسید.
`message.undelivered`	اپراتور اعلام کرد پیامک تحویل نشد.
`message.received`	پیامکی به یکی از خطوط شما رسید.
`bulk.completed` / `bulk.failed`	یک ارسال انبوه تمام شد / متوقف شد.
`payment.credited`	اعتبار حساب شارژ شد.

بررسی امضا — حتماً انجام دهید

هر درخواست وبهوک این هدرها را دارد. با بررسی امضا مطمئن می‌شوید درخواست واقعاً از ELLSMS آمده و دست‌کاری نشده است:

```
HEADERS

X-ELLSMS-Event-ID:  52bbf726-b067-4443-8a5c-ea4fe11a4f8b
X-ELLSMS-Timestamp: 1754392345
X-ELLSMS-Signature: 3f29a1…   (hex HMAC-SHA256)
```

```
FORMULA

signature = hex( HMAC_SHA256( secret, timestamp + "." + raw_body ) )
```

```php
<?php
function verify_ellsms_webhook(string $secret, string $timestamp, string $rawBody, string
$signature): bool {
    // reject stale or replayed deliveries (5 minutes)
    if (!ctype_digit($timestamp) || abs(time() - (int)$timestamp) > 300) return false;
    $expected = hash_hmac('sha256', $timestamp . '.' . $rawBody, $secret);
    return hash_equals($expected, $signature);   // constant-time
}
```

نکته‌ها

`raw_body` باید همان بایت‌هایی باشد که دریافت کرده‌اید، پیش از هر `json_decode` . سریع با کد `2xx` پاسخ دهید و پردازش سنگین را بعداً انجام دهید. با `X-ELLSMS-Event-ID` رویدادهای تکراری را تشخیص دهید.

۱۰ محدودیت‌ها

مورد	مقدار
نرخ درخواست برای هر کلید	۶۰ درخواست در دقیقه، با تحمل جهش تا ۱۵ درخواست
گیرنده در هر `POST /messages`	حداکثر ۱۰۰
طول متن پیام	حداکثر ۲۰۰۰ نویسه
ردیف در هر `POST /bulk-jobs`	حداکثر ۵۰۰۰
حجم بدنه‌ی درخواست	حداکثر ۲۵۶ کیلوبایت
اعتبار `client_message_id`	۲۴ ساعت

پلن اشتراک سازمان ممکن است این سقف‌ها را پایین‌تر بیاورد، ولی هرگز بالاتر نمی‌برد.

اگر از نرخ مجاز بیشتر بفرستید، پاسخ `429` با هدر `Retry-After` برمی‌گردد که می‌گوید چند ثانیه صبر کنید.

۱۱ خطاها

همه‌ی خطاها یک قالب ثابت دارند. فیلد `code` را در کد خود بررسی کنید؛ `message` برای انسان است و ممکن است تغییر کند.

ERROR · 422

```
{
  "error": {
    "code": "validation_failed",
    "message": "Request validation failed.",
    "request_id": "593da5ec9575f879",
    "fields": { "destinations": ["required"], "content": ["too_long — max 2000 characters"] }
  }
}
```

HTTP	code	یعنی چه، چه کار کنید
400	`invalid_request`	درخواست بدشکل است (مثلاً JSON نامعتبر یا هدر اجباری جاافتاده).
401	`unauthenticated`	کلید نیست، اشتباه است، باطل شده یا منقضی شده.
402	`subscription_inactive`	اشتراک سازمان فعال نیست.
403	`forbidden`	کلید دسترسی لازم را ندارد، یا آی‌پی شما مجاز نیست.
403	`feature_not_available`	این قابلیت در پلن فعلی نیست.
404	`not_found`	مسیر یا شناسه وجود ندارد.
409	`conflict`	شناسه‌ی ضدتکرار با بدنه‌ی دیگری استفاده شده، یا درخواست قبلی هنوز در جریان است.
413	`payload_too_large`	بدنه بیش از ۲۵۶ کیلوبایت است؛ آن را چند قسمت کنید.
422	`validation_failed`	فیلدی ایراد دارد؛ جزئیات در `fields` . مثلاً `originator: not_allowed` یعنی این خط به شما تعلق ندارد.
429	`rate_limited`	بیش از حد مجاز درخواست زدید؛ به اندازه‌ی `Retry-After` صبر کنید.
429	`quota_exceeded`	سهمیه‌ی پیامک دوره‌ی فعلی تمام شده.
500	`internal_error`	خطای سمت سرور؛ با `request_id` به پشتیبانی اطلاع دهید.
503	`service_unavailable`	سرویس موقتاً در دسترس نیست؛ کمی بعد تلاش کنید.

همیشه `request_id` را لاگ کنید

اگر به مشکلی برخوردید، همین شناسه را برای پشتیبانی بفرستید تا درخواست شما دقیقاً پیدا شود.

۱۲ نمونه کد

PHP

```php
<?php
$ch = curl_init('https://ellsms.ir/api/v1/messages');
curl_setopt_array($ch, [
    CURLOPT_POST           => true,
    CURLOPT_RETURNTRANSFER => true,
    CURLOPT_HTTPHEADER     => [
        'Authorization: Bearer ' . getenv('ELLSMS_KEY'),
        'Content-Type: application/json',
    ],
    CURLOPT_POSTFIELDS     => json_encode([
        'destinations'      => ['989121234567'],
        'content'           => 'کد ورود شما: 4521',
        'client_message_id' => 'otp-' . $userId . '-' . time(),
    ], JSON_UNESCAPED_UNICODE),
]);
$response = json_decode(curl_exec($ch), true);
$status   = curl_getinfo($ch, CURLINFO_RESPONSE_CODE);
```

Python

```python
import os, requests

r = requests.post(
    "https://ellsms.ir/api/v1/messages",
    headers={"Authorization": f"Bearer {os.environ['ELLSMS_KEY']}"},
    json={"destinations": ["989121234567"], "content": "کد ورود شما: 4521"},
    timeout=30,
)
if r.status_code == 201:
    print(r.json()["data"]["status"])
else:
    err = r.json()["error"]
    print(err["code"], err["request_id"])
```

JavaScript (Node.js)

```
NODE.JS
const res = await fetch("https://ellsms.ir/api/v1/messages", {
  method: "POST",
  headers: {
    "Authorization": `Bearer ${process.env.ELLSMS_KEY}`,
    "Content-Type": "application/json",
  },
  body: JSON.stringify({ destinations: ["989121234567"], content: "کد ورود شما: 4521" }),
});
const body = await res.json();
if (!res.ok) console.error(body.error.code, body.error.request_id);
```

۱۳ کتابخانه‌ها و منابع

برای اینکه درخواست‌ها را دستی نسازید، کتابخانه‌ی رسمی برای سه زبان آماده است. همه‌ی موارد این راهنما — احراز هویت، ضدتکرار و قالب خطا — در آن‌ها پیاده شده‌اند:

زبان	محل
PHP	`sdk/php`
JavaScript / TypeScript	`sdk/js`
Python	`sdk/python`

مشخصات OpenAPI

توضیح ماشین‌خوان کل API در فایل `openapi-v1.yaml` موجود است. می‌توانید آن را مستقیم در Postman یا Insomnia وارد کنید یا با ابزارهای تولید کد، کلاینت هر زبانی را بسازید.

چک‌لیست پیش از رفتن به محیط واقعی

- کلید در متغیر محیطی سرور است، نه در کد یا مخزن.
- هر کلید فقط دسترسی‌های لازم را دارد.
- برای ارسال‌های مهم `client_message_id` می‌فرستید.
- خطای `429` را با صبر به اندازه‌ی `Retry-After` مدیریت می‌کنید.
- `request_id` خطاها را لاگ می‌کنید.
- اگر وبهوک دارید، امضای هر درخواست را بررسی می‌کنید.